

Priority Algorithms: Unsupervised Learning

This guide covers high-priority unsupervised learning topics for ML placements, AI engineer interviews, research internships, and project work. Each topic follows the same structure: intuition, math, implementation, mistakes, limitations, interview questions, and projects.

K-Means Clustering

1. Overview

K-Means is a centroid-based clustering algorithm that partitions unlabeled data into **K** groups. It is useful when you expect compact, roughly spherical clusters and want a simple, fast baseline.

Real-world uses include customer segmentation, image color quantization, document grouping after embeddings, feature engineering, and data exploration.

2. Intuition

Imagine placing **K** magnets on a table of data points. Each point attaches to the nearest magnet. Then each magnet moves to the average position of its attached points. Repeat until the magnets stop moving.

Example: customers can be clustered by annual income and spending score into low, medium, and high-value groups.

3. Prerequisites

- Euclidean distance
- Mean and variance
- Feature scaling
- Basic NumPy and scikit-learn
- Understanding of local minima

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Centroid	Mean vector of a cluster	Defines cluster center	Average customer profile	Why mean makes K-Means sensitive to outliers
Assignment step	Assign each point to nearest centroid	Creates clusters	Point joins closest group	Distance metric assumptions
Update step	Recompute centroids	Improves fit	New cluster average	Why objective decreases

Concept	Meaning	Why It Matters	Example	Interview Angle
Inertia/WCSS	Sum of squared distances to centroids	Main objective	Lower inertia means tighter clusters	Elbow method
Initialization	Starting centroid choice	Affects final clusters	Random vs k-means++	Local optimum problem

5. Algorithm / Working Process

Input: feature matrix X and number of clusters K .

Steps:

1. Initialize K centroids.
2. Assign every point to the nearest centroid.
3. Recompute each centroid as the mean of assigned points.
4. Repeat assignment and update until convergence.
5. Output cluster labels and centroid locations.

Training is unsupervised because labels are not used. Inference assigns a new point to its nearest learned centroid.

6. Mathematical Foundation

K-Means minimizes within-cluster sum of squares:

$$J = \sum_{i=1}^n \sum_{k=1}^K z_{ik} \|x_i - \mu_k\|^2$$

where $z_{ik} = 1$ if point x_i belongs to cluster k , otherwise 0 , and μ_k is centroid k .

Centroid update:

$$\mu_k = (1 / |C_k|) \sum_{x_i \in C_k} x_i$$

The algorithm monotonically decreases the objective but can converge to a local minimum.

7. Practical Implementation

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

```
X, _ = make_blobs(n_samples=500, centers=4, cluster_std=1.2, random_state=42)
X = StandardScaler().fit_transform(X)

model = KMeans(n_clusters=4, init="k-means++", n_init=10, random_state=42)
labels = model.fit_predict(X)

print("Centroids:\n", model.cluster_centers_)
print("Inertia:", model.inertia_)
print("Silhouette:", silhouette_score(X, labels))
```

8. Code Explanation

`make_blobs` creates synthetic clustered data. `StandardScaler` prevents features with large numeric ranges from dominating distance. `KMeans` learns centroids and labels. `inertia_` measures compactness, while silhouette checks whether clusters are separated.

9. Training / Evaluation

There is no train/test split requirement for pure clustering, but you can split if using clusters as downstream features. Common metrics include inertia, silhouette score, Davies-Bouldin index, Calinski-Harabasz score, and domain validation.

Important hyperparameters: `n_clusters`, `init`, `n_init`, `max_iter`, `tol`.

Improve performance by scaling features, removing outliers, using PCA for noisy high-dimensional data, and trying different `K` values.

10. Complexity and Cost

Training complexity is approximately:

$$O(n * K * d * i)$$

where `n` is samples, `d` is dimensions, and `i` is iterations. Inference is $O(Kd)$ per point. Memory is $O(n + Kd)$. CPU is usually enough.

11. Common Use Cases

- Customer segmentation
- Image compression
- Document clustering over embeddings
- Market basket user grouping
- Geospatial region grouping
- Prototype-based feature engineering

12. Common Mistakes

- Forgetting feature scaling

- Choosing K only by inertia
- Using K-Means on non-spherical clusters
- Ignoring outliers
- Treating cluster IDs as ordered labels
- Assuming clusters are stable across random seeds
- Using K-Means on categorical data without proper encoding

13. Edge Cases / Limitations

K-Means performs poorly with non-convex clusters, unequal density, heavy outliers, many categorical variables, and high-dimensional sparse data. It also requires K in advance.

14. Variations

Variation	What Changes	When To Use	Placement Importance
MiniBatch K-Means	Updates centroids using batches	Large datasets	High
K-Medoids	Uses real points as centers	Outlier robustness	Medium
Kernel K-Means	Clusters in implicit feature space	Nonlinear clusters	Medium
Spherical K-Means	Uses cosine similarity	Text embeddings	High

15. Related Topics

K-Means relates to GMMs because both use cluster centers, but GMMs give soft probabilistic assignments. It relates to PCA because PCA can reduce dimensions before clustering. It differs from DBSCAN because K-Means assumes compact clusters and requires K .

16. Interview Questions

1. What does K-Means optimize? It minimizes within-cluster squared distance to centroids.
2. Why scale features? Distance-based algorithms are dominated by large-scale features.
3. Why can K-Means converge to different answers? Random initialization can lead to different local minima.
4. What is k-means++? A smarter initialization that spreads initial centroids.
5. How do you choose K ? Elbow, silhouette, gap statistic, and domain usefulness.
6. Why is K-Means sensitive to outliers? Centroids are means, and means shift under extreme values.
7. Can K-Means handle categorical data? Not directly; use proper encodings or k-modes.
8. What happens if a cluster becomes empty? Implementations reinitialize or handle the empty centroid.
9. Is K-Means supervised? No, it uses no target labels.
10. K-Means vs GMM? K-Means gives hard spherical clusters; GMM gives soft elliptical probabilistic clusters.

17. Practice Tasks

- Implement K-Means from scratch using NumPy.
- Cluster Mall Customers dataset and explain segments.
- Compare elbow and silhouette for $K=2..10$.

- Debug a result where one feature dominates because scaling was skipped.
- Extend with PCA visualization.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Customer Segmentation	Groups users by spending behavior	pandas, sklearn	Mall Customers	Business ML
Image Color Compressor	Reduces image colors to K centroids	OpenCV, sklearn	Any images	Visual ML
Embedding Cluster Explorer	Clusters sentence embeddings	sentence-transformers, sklearn	News/articles	NLP + unsupervised

19. Quick Revision

- Key idea: assign points to nearest centroid, update centroids.
- Main formula: minimize $\sum ||x_i - \mu_k||^2$.
- When to use: compact numeric clusters.
- Metrics: inertia, silhouette.
- Common traps: no scaling, wrong **K**, outliers.
- Interview one-liner: K-Means is fast centroid-based clustering for spherical groups.

20. Final Cheat Sheet

Item	Summary
Definition	Partitions data into K centroid-based clusters
Input/output	X -> cluster labels and centroids
Main steps	initialize, assign, update, repeat
Hyperparameters	n_clusters , init , n_init , max_iter
Metrics	inertia, silhouette
Pros	simple, fast, scalable
Cons	needs K , sensitive to scale/outliers
Best use	quick segmentation baseline

Hierarchical Clustering

1. Overview

Hierarchical clustering builds a tree of clusters instead of forcing one fixed number of clusters immediately. The tree is called a dendrogram. It is useful when you want to inspect cluster structure at multiple granularities.

It is used in biology, document grouping, customer segmentation, taxonomy discovery, and exploratory analysis.

2. Intuition

Think of arranging people into friend groups. At first everyone is alone. Then the two most similar people merge. Next, similar pairs or groups merge. Eventually everyone belongs to one big group. Cutting the tree at a chosen height gives clusters.

3. Prerequisites

- Distance metrics
- Linkage criteria
- Matrix operations
- Basic clustering metrics
- Dendrogram interpretation

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Agglomerative	Bottom-up merging	Most common approach	Start with each point alone	Complexity
Divisive	Top-down splitting	Less common	Start with all points	Harder optimization
Linkage	Distance between clusters	Controls cluster shape	Single vs complete	Chaining problem
Dendrogram	Tree visualization	Helps choose cluster count	Cut at height 5	Interpretability
Distance metric	Similarity definition	Drives all merges	Euclidean/cosine	Metric choice

5. Algorithm / Working Process

Input: data matrix X , distance metric, linkage method.

Steps:

1. Treat each point as its own cluster.
2. Compute pairwise distances between clusters.
3. Merge the two closest clusters.
4. Update distances using linkage.
5. Repeat until one cluster remains or desired cluster count is reached.

6. Cut the dendrogram to obtain final labels.

6. Mathematical Foundation

Common linkage formulas:

```
Single linkage:  d(A, B) = min d(a, b)
Complete linkage: d(A, B) = max d(a, b)
Average linkage: d(A, B) = mean d(a, b)
Ward linkage:    merge that minimizes increase in within-cluster variance
```

Ward's method approximately minimizes:

$$\Delta(A, B) = (|A| \cdot |B| / (|A| + |B|)) \cdot \|\mu_A - \mu_B\|^2$$

7. Practical Implementation

```
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score

X, _ = make_blobs(n_samples=300, centers=3, random_state=7)
X = StandardScaler().fit_transform(X)

model = AgglomerativeClustering(n_clusters=3, linkage="ward")
labels = model.fit_predict(X)

print("Silhouette:", silhouette_score(X, labels))
print("First labels:", labels[:10])
```

8. Code Explanation

The code creates numeric cluster data, scales it, then applies agglomerative clustering. `linkage="ward"` is a strong default for Euclidean numeric data because it favors compact clusters.

9. Training / Evaluation

Evaluation is similar to other clustering methods: silhouette score, dendrogram inspection, cluster stability, and domain review. There is no gradient training. Hyperparameters include `n_clusters`, `distance_threshold`, `metric`, and `linkage`.

10. Complexity and Cost

Hierarchical clustering usually needs pairwise distances, so memory is often $O(n^2)$. Time complexity is commonly $O(n^2 \log n)$ or worse depending on implementation. It is not ideal for very large datasets.

11. Common Use Cases

- Gene expression clustering
- Topic hierarchy discovery
- Customer segmentation with dendrogram explainability
- Document clustering
- Taxonomy generation

12. Common Mistakes

- Using it on huge datasets without sampling
- Choosing linkage blindly
- Misreading dendrogram height
- Forgetting scaling
- Using Ward linkage with non-Euclidean metrics
- Assuming dendrogram cut is objectively correct

13. Edge Cases / Limitations

It struggles with large data, noisy distances, high-dimensional data, and irreversible early merges. Single linkage can create long chained clusters.

14. Variations

Variation	What Changes	When To Use	Importance
Agglomerative	Bottom-up	Standard use	High
Divisive	Top-down	Taxonomy splitting	Medium
BIRCH	Clustering feature tree	Large data	Medium
Ward clustering	Variance-minimizing merges	Compact numeric clusters	High

15. Related Topics

Compared with K-Means, hierarchical clustering does not require choosing K before building the tree. Compared with DBSCAN, it does not explicitly model noise. Compared with spectral clustering, it is simpler but less powerful for graph-like structures.

16. Interview Questions

1. What is a dendrogram? A tree showing cluster merges and merge distances.
2. Agglomerative vs divisive? Agglomerative merges bottom-up; divisive splits top-down.
3. What is linkage? A rule for distance between clusters.
4. Why is single linkage risky? It can chain points through bridges.
5. What does Ward linkage optimize? It minimizes increase in within-cluster variance.

6. Does it need **K**? Not before building; final clusters come from a cut.
7. Why is it expensive? Pairwise distances need large time and memory.
8. How choose clusters? Cut dendrogram, use distance threshold, or metrics.
9. Is scaling needed? Yes for distance-based numeric clustering.
10. When prefer hierarchical over K-Means? When cluster hierarchy and interpretability matter.

17. Practice Tasks

- Plot a dendrogram with SciPy.
- Compare single, complete, average, and Ward linkage.
- Cluster document embeddings using cosine distance.
- Debug chained clusters under single linkage.
- Use a distance threshold instead of fixed **n_clusters**.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Research Paper Taxonomy	Groups papers into topic tree	sklearn, scipy	arXiv metadata	Research tooling
Product Category Discovery	Builds category hierarchy	pandas, sklearn	E-commerce catalog	Business ML
Gene Cluster Explorer	Clusters gene expression profiles	scipy, seaborn	UCI gene data	Bioinformatics

19. Quick Revision

- Key idea: build cluster tree using repeated merges.
- Main formula: linkage distance.
- When to use: small/medium data needing hierarchy.
- Metrics: silhouette, dendrogram height.
- Common traps: scaling, linkage mismatch, high cost.
- Interview one-liner: hierarchical clustering gives a multi-resolution cluster tree.

20. Final Cheat Sheet

Item	Summary
Definition	Tree-based clustering
Input/output	X -> dendrogram/labels
Main steps	compute distances, merge closest clusters
Hyperparameters	linkage, metric, n_clusters , threshold
Metrics	silhouette, dendrogram inspection
Pros	interpretable hierarchy

Item	Summary
Cons	expensive, early mistakes irreversible
Best use	taxonomy and exploratory clustering

DBSCAN

1. Overview

DBSCAN means Density-Based Spatial Clustering of Applications with Noise. It finds dense regions separated by sparse regions and marks isolated points as noise. It does not require pre-selecting the number of clusters.

It is useful for geospatial clustering, anomaly removal, sensor data, fraud pre-filtering, and non-spherical cluster discovery.

2. Intuition

Imagine dots on a map. A cluster is an area where each dot has enough nearby neighbors. Sparse dots outside dense areas are treated as noise.

3. Prerequisites

- Distance metrics
- Nearest neighbors
- Density intuition
- Feature scaling
- Clustering metrics

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
eps	Neighborhood radius	Defines local density	Points within 0.5 units	Hardest hyperparameter
min_samples	Minimum neighbors for core point	Controls density threshold	At least 5 neighbors	Noise sensitivity
Core point	Dense point	Expands clusters	In crowded area	Cluster formation
Border point	Near core but not dense itself	Belongs to cluster	Edge of group	Assignment ambiguity
Noise point	Not density-reachable	Outlier	Isolated transaction	Anomaly detection

5. Algorithm / Working Process

Input: X , eps , $min_samples$.

Steps:

1. For each point, find neighbors within `eps`.
2. Mark point as core if neighbors $\geq$ `min_samples`.
3. Start from an unvisited core point.
4. Expand cluster through density-reachable core points.
5. Assign border points to clusters.
6. Mark unreachable points as noise label `-1`.

6. Mathematical Foundation

Epsilon neighborhood:

$$N_{\text{eps}}(x) = \{y \text{ in } X : \text{distance}(x, y) \leq \text{eps}\}$$

Core condition:

$$|N_{\text{eps}}(x)| \geq \text{min_samples}$$

Density reachability means `q` is reachable from `p` through a chain of core points.

7. Practical Implementation

```
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score

X, _ = make_moons(n_samples=500, noise=0.07, random_state=42)
X = StandardScaler().fit_transform(X)

model = DBSCAN(eps=0.25, min_samples=5)
labels = model.fit_predict(X)

non_noise = labels != -1
print("Clusters:", len(set(labels)) - (1 if -1 in labels else 0))
print("Noise points:", (~non_noise).sum())
print("Silhouette:", silhouette_score(X[non_noise], labels[non_noise]))
```

8. Code Explanation

`make_moons` creates non-spherical clusters where K-Means often fails. DBSCAN discovers curved dense regions. Noise points get label `-1`, so they are excluded from silhouette scoring.

9. Training / Evaluation

DBSCAN does not train parameters. It computes neighborhoods and cluster expansion. Use k-distance plots to tune **eps**, and domain knowledge for **min_samples**. Evaluation uses noise ratio, cluster count, silhouette on non-noise points, and manual inspection.

10. Complexity and Cost

With indexing, DBSCAN is often near $O(n \log n)$ for low-dimensional data. Without efficient neighbor search, it can be $O(n^2)$. High-dimensional distances reduce effectiveness.

11. Common Use Cases

- GPS location clustering
- Fraud/anomaly pre-filtering
- Duplicate entity grouping
- Sensor event clustering
- Shape-based clustering

12. Common Mistakes

- Not scaling features
- Setting **eps** too large and merging everything
- Setting **eps** too small and labeling everything noise
- Using DBSCAN in very high dimensions without embeddings/reduction
- Comparing labels directly across runs or datasets
- Ignoring variable-density clusters

13. Edge Cases / Limitations

DBSCAN struggles when clusters have different densities, distances become meaningless in high dimensions, or data has no clear density gap.

14. Variations

Variation	What Changes	When To Use	Importance
HDBSCAN	Hierarchical density clustering	Variable density	High
OPTICS	Orders points by density reachability	Unknown eps	Medium
ST-DBSCAN	Adds spatial-temporal constraints	GPS/time data	Medium

15. Related Topics

DBSCAN contrasts with K-Means because it finds arbitrary shapes and noise. It connects to anomaly detection because noise labels can be treated as outliers. HDBSCAN is a stronger practical extension.

16. Interview Questions

1. What does DBSCAN stand for? Density-Based Spatial Clustering of Applications with Noise.
2. Does DBSCAN need **K**? No.

3. What is a core point? A point with at least `min_samples` neighbors inside `eps`.
4. What is a border point? A non-core point reachable from a core point.
5. What label marks noise in sklearn? `-1`.
6. How choose `eps`? Use k-distance plot and domain knowledge.
7. Why scale data? Neighborhood distances depend on feature scales.
8. DBSCAN vs K-Means? DBSCAN handles noise and arbitrary shapes; K-Means is faster for spherical clusters.
9. Main weakness? Variable density and high dimensions.
10. Can DBSCAN predict new points naturally? Standard DBSCAN does not learn a centroid model; use approximate assignment if needed.

17. Practice Tasks

- Cluster two-moons data.
- Draw a k-distance plot for `eps`.
- Compare DBSCAN and K-Means on non-spherical data.
- Analyze noise points as anomalies.
- Try HDBSCAN on variable-density data.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Hotspot Finder	Finds dense event locations	sklearn, geopandas	NYC taxi	Geospatial ML
Fraud Burst Detector	Finds dense suspicious events and outliers	pandas, sklearn	Credit card fraud	Risk analytics
Sensor Event Grouper	Groups machine events	sklearn	NASA bearing/sensor data	Industrial AI

19. Quick Revision

- Key idea: dense regions are clusters; sparse points are noise.
- Main formula: $|N_{eps}(x)| \geq min_samples$.
- When to use: arbitrary shapes with outliers.
- Metrics: noise ratio, silhouette, visual checks.
- Common traps: bad `eps`, no scaling.
- Interview one-liner: DBSCAN clusters by density and identifies noise automatically.

20. Final Cheat Sheet

Item	Summary
Definition	Density-based clustering
Input/output	<code>X</code> -> labels with <code>-1</code> noise

Item	Summary
Main steps	find neighborhoods, expand core points
Hyperparameters	<code>eps</code> , <code>min_samples</code> , metric
Metrics	cluster count, noise rate, silhouette
Pros	arbitrary shapes, no <code>K</code> , detects noise
Cons	sensitive to density settings
Best use	spatial/noisy clustering

PCA

1. Overview

Principal Component Analysis reduces dimensionality by projecting data onto directions of maximum variance. It is one of the most important unsupervised techniques for compression, denoising, visualization, and preprocessing.

2. Intuition

If a cloud of points forms a stretched ellipse, PCA finds the longest direction first. That direction captures the most variation. The second direction is perpendicular and captures the next most variation.

3. Prerequisites

- Linear algebra: vectors, matrices, eigenvectors
- Variance and covariance
- Standardization
- Matrix decomposition
- Basic visualization

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Principal component	Direction of maximum variance	New feature axis	PC1 of height/weight	Eigenvectors
Explained variance	Variance captured by each PC	Choose dimensions	95% variance retained	Scree plot
Orthogonality	PCs are perpendicular	Removes redundancy	PC1 independent direction from PC2	Decorrelation
Projection	Mapping data to PCs	Dimensionality reduction	100D to 2D	Information loss

Concept	Meaning	Why It Matters	Example	Interview Angle
Reconstruction	Mapping back approximately	Compression quality	Image compression	Error tradeoff

5. Algorithm / Working Process

Input: numeric matrix X .

Steps:

1. Center features by subtracting mean.
2. Usually scale features to unit variance.
3. Compute covariance matrix or use SVD.
4. Find eigenvectors/eigenvalues.
5. Sort components by eigenvalue.
6. Project data onto top k components.

Output: lower-dimensional representation and components.

6. Mathematical Foundation

Covariance matrix:

$$\text{Sigma} = (1 / (n - 1)) X_{\text{centered}}^T X_{\text{centered}}$$

Eigen decomposition:

$$\text{Sigma} v_j = \lambda_j v_j$$

Projection:

$$Z = X_{\text{centered}} W_k$$

Explained variance ratio:

$$\lambda_j / \sum_i \lambda_i$$

PCA minimizes reconstruction error among all linear projections of dimension k .

7. Practical Implementation

```
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score

X, y = load_digits(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

pipe = make_pipeline(
    StandardScaler(),
    PCA(n_components=0.95, random_state=42),
    LogisticRegression(max_iter=2000)
)

pipe.fit(X_train, y_train)
pred = pipe.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("PCA components:", pipe.named_steps["pca"].n_components_)
```

8. Code Explanation

The pipeline prevents leakage by fitting scaling and PCA only on training data. `n_components=0.95` keeps enough components to explain 95% of variance. Logistic regression then trains on compressed features.

9. Training / Evaluation

For preprocessing, fit PCA on training data only. Evaluate downstream task performance, explained variance, reconstruction error, or visualization usefulness. Important hyperparameters: `n_components`, whitening, solver.

10. Complexity and Cost

Full PCA via SVD can cost about $O(\min(n \cdot d^2, d \cdot n^2))$. Memory depends on `X` and components. Randomized PCA is better for large data. CPU is enough for common tabular datasets.

11. Common Use Cases

- Dimensionality reduction
- Noise reduction
- 2D/3D visualization
- Feature decorrelation
- Compression
- Preprocessing before clustering

12. Common Mistakes

- Applying PCA before train/test split
- Not scaling features
- Assuming high variance always means high predictive value
- Using PCA for nonlinear manifolds
- Interpreting PCs as original features without checking loadings
- Keeping too few components

13. Edge Cases / Limitations

PCA is linear, sensitive to outliers, and variance-focused rather than label-focused. It can hurt performance if low-variance features are predictive.

14. Variations

Variation	What Changes	When To Use	Importance
Kernel PCA	Nonlinear projection through kernels	Nonlinear data	Medium
Incremental PCA	Batch updates	Large data	Medium
Sparse PCA	Sparse component loadings	Interpretability	Medium
Randomized PCA	Approximate fast SVD	Large matrices	High
Whitening	Scales PCs to unit variance	Certain models	Medium

15. Related Topics

PCA differs from t-SNE and UMAP because PCA is linear and preserves global variance. PCA is often used before K-Means, GMMs, or visualization. Autoencoders can be seen as nonlinear learned dimensionality reduction.

16. Interview Questions

1. What does PCA do? Projects data onto directions of maximum variance.
2. Why center data? PCA directions depend on covariance around the mean.
3. Why scale data? Large-scale features dominate variance.
4. What are principal components? Eigenvectors of covariance matrix.
5. What do eigenvalues represent? Variance explained by components.
6. Is PCA supervised? No.
7. Can PCA improve accuracy? Sometimes, by denoising; sometimes it removes useful signal.
8. PCA vs LDA? PCA is unsupervised variance maximization; LDA is supervised class separation.
9. What is explained variance ratio? Fraction of total variance captured by a component.
10. Why use SVD? It is numerically stable and avoids explicitly forming covariance.

17. Practice Tasks

- Implement PCA with NumPy eigen decomposition.
- Compress digit images and reconstruct them.

- Plot explained variance curve.
- Compare classifier performance before and after PCA.
- Debug data leakage in PCA preprocessing.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Image Compressor	Reconstructs images from top PCs	sklearn, matplotlib	Olivetti faces	Linear algebra + CV
PCA Cluster Dashboard	Visualizes clusters in 2D	streamlit, sklearn	Customer data	Explainable ML
Noise Reduction Demo	Removes noise via PCA reconstruction	numpy, sklearn	Digits	Signal processing

19. Quick Revision

- Key idea: keep directions with maximum variance.
- Main formula: $\Sigma \mathbf{v} = \lambda \mathbf{v}$.
- When to use: compression, denoising, visualization.
- Metrics: explained variance, reconstruction error.
- Common traps: leakage, no scaling, overinterpretation.
- Interview one-liner: PCA is linear dimensionality reduction using eigenvectors of covariance.

20. Final Cheat Sheet

Item	Summary
Definition	Linear dimensionality reduction
Input/output	high-dimensional $\mathbf{X}$ -> lower-dimensional $\mathbf{Z}$
Main steps	center, decompose, project
Hyperparameters	<code>n_components</code> , whitening, solver
Metrics	explained variance, reconstruction error
Pros	fast, interpretable, useful preprocessing
Cons	linear, outlier-sensitive
Best use	compression and visualization baseline

t-SNE Intuition

1. Overview

t-SNE, or t-distributed Stochastic Neighbor Embedding, is a nonlinear visualization algorithm mainly used to map high-dimensional data into 2D or 3D while preserving local neighborhoods.

It is popular for visualizing embeddings, image features, gene expression data, and hidden neural network representations.

2. Intuition

t-SNE tries to keep close neighbors close. If two points are similar in high-dimensional space, t-SNE strongly prefers them to remain near each other in the 2D map. It cares more about local neighborhoods than exact global distances.

3. Prerequisites

- Probability distributions
- Pairwise distances
- KL divergence
- Gradient descent intuition
- Embeddings and dimensionality reduction

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Local similarity	Probability of neighbor relation	Preserves neighborhoods	Similar images nearby	Not a clustering algorithm
Perplexity	Effective neighborhood size	Controls local/global balance	5 vs 50 neighbors	Tuning
Student-t distribution	Heavy-tailed low-dimensional similarity	Avoids crowding	Separates clusters visually	Crowding problem
KL divergence	Objective mismatch measure	Optimization target	High-D vs low-D probabilities	Asymmetry
Random seed	Initialization sensitivity	Different layouts possible	Rotated/scattered maps	Reproducibility

5. Algorithm / Working Process

Input: high-dimensional features or embeddings.

Steps:

1. Compute pairwise similarities in high-dimensional space.
2. Convert similarities into probabilities p_{ij} .
3. Initialize low-dimensional points.
4. Compute low-dimensional similarities q_{ij} using Student-t distribution.

5. Minimize KL divergence between **P** and **Q**.
6. Output 2D/3D coordinates for visualization.

t-SNE is usually not used for inference on new points.

6. Mathematical Foundation

High-dimensional conditional probability:

$$p_{\{j|i\}} = \exp(-||x_i - x_j||^2 / 2 \sigma_i^2) / \sum_{k \neq i} \exp(-||x_i - x_k||^2 / 2 \sigma_i^2)$$

Low-dimensional similarity:

$$q_{ij} = (1 + ||y_i - y_j||^2)^{-1} / \sum_{k \neq i} (1 + ||y_i - y_k||^2)^{-1}$$

Objective:

$$KL(P || Q) = \sum_i \sum_j p_{ij} \log(p_{ij} / q_{ij})$$

7. Practical Implementation

```
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

X, y = load_digits(return_X_y=True)
X = StandardScaler().fit_transform(X)

# PCA first speeds up t-SNE and removes some noise.
X50 = PCA(n_components=50, random_state=42).fit_transform(X)

tsne = TSNE(
    n_components=2,
    perplexity=30,
    learning_rate="auto",
    init="pca",
    random_state=42
)
coords = tsne.fit_transform(X50)

print(coords[:5])
print(y[:5])
```

8. Code Explanation

Digits are scaled, reduced with PCA, then embedded with t-SNE. PCA is a practical preprocessing step because t-SNE is expensive on high-dimensional noisy data. The output coordinates are for plotting, not classification by themselves.

9. Training / Evaluation

t-SNE does not train a reusable predictive model in standard sklearn usage. Evaluate visually, by neighborhood preservation, and by stability across seeds/perplexities. Avoid using visual clusters as proof of real classes without validation.

10. Complexity and Cost

Exact t-SNE is expensive, around $O(n^2)$. Barnes-Hut or FFT approximations improve scaling. CPU works for thousands of points; large datasets need sampling or faster implementations.

11. Common Use Cases

- Visualizing embeddings
- Inspecting class separability
- Analyzing neural network features
- Bioinformatics visualization
- Debugging representation learning

12. Common Mistakes

- Treating t-SNE as a clustering algorithm
- Interpreting distances between far clusters literally
- Ignoring perplexity sensitivity
- Running directly on raw unscaled features
- Comparing axes as meaningful dimensions
- Overclaiming from pretty plots

13. Edge Cases / Limitations

t-SNE is slow, stochastic, weak at global structure, and awkward for new unseen points. It can create visually separated groups even when the structure is less clear.

14. Variations

Variation	What Changes	When To Use	Importance
Barnes-Hut t-SNE	Approximate nearest interactions	Medium datasets	High
Flt-SNE/openTSNE	Faster approximation	Large visualization	Medium
Parametric t-SNE	Neural net maps points	Need transform for new data	Research

15. Related Topics

t-SNE is related to UMAP, which is often faster and preserves more global structure. PCA is usually used before t-SNE. Unlike clustering methods, t-SNE only provides coordinates for visualization.

16. Interview Questions

1. What is t-SNE used for? High-dimensional visualization.
2. Does t-SNE preserve global distances? Not reliably.
3. What is perplexity? A neighborhood-size parameter.
4. Why use Student-t distribution? Heavy tails reduce crowding.
5. What loss does it minimize? KL divergence between similarity distributions.
6. Should we cluster t-SNE output? Be careful; t-SNE can distort distances.
7. Why run PCA before t-SNE? Speed and denoising.
8. Are axes meaningful? No, orientation and axis values are arbitrary.
9. Can t-SNE transform new points? Standard t-SNE does not naturally support this.
10. t-SNE vs PCA? t-SNE is nonlinear/local; PCA is linear/global variance.

17. Practice Tasks

- Visualize MNIST/digits embeddings.
- Compare perplexity values 5, 30, 50.
- Run t-SNE with and without PCA preprocessing.
- Check stability across random seeds.
- Compare t-SNE and UMAP on the same embeddings.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Embedding Map	Visualizes sentence embeddings	sklearn, sentence-transformers	News titles	NLP interpretability
CNN Feature Explorer	Shows image feature clusters	PyTorch, sklearn	CIFAR-10	Deep learning analysis
Bio Cell Visualizer	Maps gene expression data	scanpy/sklearn	PBMC data	Research flavor

19. Quick Revision

- Key idea: preserve local neighbors in 2D.
- Main formula: minimize $KL(P || Q)$.
- When to use: visualization.
- Metrics: visual quality, trustworthiness.
- Common traps: overinterpreting clusters and distances.
- Interview one-liner: t-SNE is a nonlinear local-neighborhood visualization method.

20. Final Cheat Sheet

Item	Summary
Definition	Nonlinear dimensionality reduction for visualization
Input/output	embeddings/features -> 2D coordinates
Main steps	compute similarities, optimize low-D map
Hyperparameters	perplexity, learning rate, iterations
Metrics	trustworthiness, visual stability
Pros	excellent local visual separation
Cons	slow, stochastic, weak global meaning
Best use	embedding inspection

UMAP Intuition

1. Overview

UMAP, or Uniform Manifold Approximation and Projection, is a nonlinear dimensionality reduction method used for visualization and sometimes preprocessing. It often runs faster than t-SNE and can preserve more global structure.

2. Intuition

UMAP assumes high-dimensional data lies on a lower-dimensional manifold. It builds a neighbor graph in high dimensions, then tries to create a low-dimensional graph with similar neighbor relationships.

3. Prerequisites

- Nearest neighbors
- Graphs
- Distance metrics
- Embeddings
- Basic probability/objective intuition

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Manifold	Lower-dimensional structure inside high-D space	Basis of UMAP	Images vary by pose/lighting	Assumption
kNN graph	Graph of nearest neighbors	Captures local structure	Each point connected to 15 neighbors	n_neighbors

Concept	Meaning	Why It Matters	Example	Interview Angle
Fuzzy simplicial set	Weighted neighbor graph	Represents uncertainty	Strong/weak edges	High-level intuition enough
<code>min_dist</code>	Minimum spacing in embedding	Controls compactness	Tight vs spread clusters	Visualization tuning
Transform	Mapping new points	More practical than t-SNE	Embed new documents	Production angle

5. Algorithm / Working Process

Input: high-dimensional data.

Steps:

1. Find nearest neighbors.
2. Build weighted high-dimensional graph.
3. Initialize low-dimensional coordinates.
4. Optimize coordinates so low-dimensional graph resembles high-dimensional graph.
5. Return embedding.
6. Optionally transform new points using learned structure.

6. Mathematical Foundation

UMAP minimizes a fuzzy-set cross entropy between high-dimensional edge weights `v_ij` and low-dimensional edge weights `w_ij`:

$$C = \sum_{ij} [v_{ij} \log(v_{ij} / w_{ij}) + (1 - v_{ij}) \log((1 - v_{ij}) / (1 - w_{ij}))]$$

Low-dimensional similarity is modeled roughly as:

$$w_{ij} = 1 / (1 + a ||y_i - y_j||^{(2b)})$$

`n_neighbors` controls local/global balance. `min_dist` controls how tightly points pack.

7. Practical Implementation

```
# pip install umap-learn
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
import umap

X, y = load_digits(return_X_y=True)
X = StandardScaler().fit_transform(X)
```

```
reducer = umap.UMAP(  
    n_neighbors=15,  
    min_dist=0.1,  
    n_components=2,  
    metric="euclidean",  
    random_state=42  
)  
coords = reducer.fit_transform(X)  
  
new_coords = reducer.transform(X[:3])  
print(coords.shape)  
print(new_coords)
```

8. Code Explanation

The code scales digit features and learns a 2D UMAP embedding. `n_neighbors=15` balances local detail and global structure. `transform` embeds new samples using the learned manifold approximation.

9. Training / Evaluation

Evaluate UMAP visually, with trustworthiness, downstream model performance, and stability across seeds. For supervised tasks, fit UMAP only on training data to avoid leakage.

10. Complexity and Cost

UMAP uses approximate nearest neighbors and is usually faster than t-SNE for large datasets. Cost depends heavily on neighbor search. CPU is enough for many datasets.

11. Common Use Cases

- Embedding visualization
- Single-cell biology
- Document maps
- Preprocessing before clustering
- Interactive data exploration

12. Common Mistakes

- Treating visualization distances as exact
- Not scaling numeric data
- Choosing `n_neighbors` without checking stability
- Using UMAP before train/test split
- Assuming clusters are real without validation
- Ignoring random seed effects

13. Edge Cases / Limitations

UMAP can distort structure, depends on parameters, and can produce different layouts across seeds. It may preserve misleading neighborhoods if the original distance metric is poor.

14. Variations

Variation	What Changes	When To Use	Importance
Supervised UMAP	Uses labels during embedding	Better class separation	Medium
Parametric UMAP	Neural network learns mapping	Large/new data	Research
densMAP	Preserves density better	Density-sensitive visualization	Medium
AlignedUMAP	Aligns multiple embeddings	Time-series manifolds	Low/Research

15. Related Topics

UMAP is often compared with t-SNE. It is usually faster, can transform new data, and may preserve global structure better. PCA is linear and often used before UMAP for denoising.

16. Interview Questions

1. What is UMAP used for? Nonlinear dimensionality reduction and visualization.
2. UMAP vs t-SNE? UMAP is often faster and can transform new data.
3. What does `n_neighbors` control? Local vs global structure balance.
4. What does `min_dist` control? Tightness of points in low-dimensional space.
5. Is UMAP supervised? Standard UMAP is unsupervised; supervised UMAP exists.
6. Does UMAP preserve exact distances? No.
7. Why scale data? Neighbor search depends on distance.
8. Can UMAP be used before clustering? Yes, but validate because it distorts data.
9. What graph does UMAP build? A weighted nearest-neighbor graph.
10. Main risk? Overinterpreting visual clusters.

17. Practice Tasks

- Install `umap-learn` and visualize digits.
- Compare `n_neighbors=5`, `15`, and `50`.
- Compare `min_dist=0.0` and `0.8`.
- Cluster UMAP output and evaluate carefully.
- Use UMAP on sentence embeddings.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Semantic Map	Interactive map of text embeddings	UMAP, Streamlit	News headlines	NLP visualization
Cell Type Explorer	Visualizes cell populations	scanpy, UMAP	PBMC	Research internship
Product Similarity Map	Shows similar products	sklearn, UMAP	Retail products	Recommender insight

19. Quick Revision

- Key idea: preserve neighbor graph in low dimensions.
- Main formula: fuzzy graph cross entropy.
- When to use: fast nonlinear visualization.
- Metrics: trustworthiness, stability.
- Common traps: leakage and overinterpretation.
- Interview one-liner: UMAP learns a low-dimensional layout that preserves nearest-neighbor structure.

20. Final Cheat Sheet

Item	Summary
Definition	Manifold-based dimensionality reduction
Input/output	high-D data -> low-D embedding
Main steps	kNN graph, low-D graph optimization
Hyperparameters	<code>n_neighbors</code> , <code>min_dist</code> , metric
Metrics	trustworthiness, visual/domain checks
Pros	fast, strong visualizations, transform support
Cons	parameter-sensitive
Best use	embedding exploration

Anomaly Detection

1. Overview

Anomaly detection identifies rare observations that differ significantly from normal behavior. It can be unsupervised, semi-supervised, or supervised depending on label availability.

It is used in fraud detection, cybersecurity, machine failure detection, medical screening, quality control, and monitoring ML systems.

2. Intuition

If most credit card transactions are small and local, a sudden huge transaction from another country may be anomalous. The model learns what normal looks like and flags unusual cases.

3. Prerequisites

- Probability and statistics
- Distance metrics
- Classification metrics
- Imbalanced data handling
- Feature engineering

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Point anomaly	Single unusual point	Most common	Huge transaction	Detection score
Contextual anomaly	Unusual in context	Needs extra variables	High AC use in winter? maybe normal in summer	Feature design
Collective anomaly	Group pattern abnormal	Sequence/log use	Many failed logins	Time windows
Contamination	Expected anomaly fraction	Sets threshold	1% fraud	Threshold tuning
Score threshold	Converts score to label	Controls precision/recall	Flag top 0.5%	Business tradeoff

5. Algorithm / Working Process

Generic process:

1. Define normal and anomalous behavior.
2. Clean and scale features.
3. Train unsupervised detector on mostly normal data.
4. Produce anomaly scores.
5. Choose threshold using labels, expected contamination, or cost.
6. Investigate and retrain with feedback.

Output is usually an anomaly score and binary flag.

6. Mathematical Foundation

Common approaches:

```

Distance score:     $s(x) = \min \text{ distance from } x \text{ to normal cluster}$ 
Density score:      $s(x) = -\log p(x)$ 
Reconstruction:    $s(x) = ||x - \text{decoder}(\text{encoder}(x))||^2$ 
Isolation score:   shorter isolation path => more anomalous
  
```

For thresholding:

```

flag(x) = 1 if  $s(x) > \tau$  else 0
  
```

Precision and recall matter more than accuracy because anomalies are rare.

7. Practical Implementation

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest

X_normal, _ = make_blobs(n_samples=500, centers=1, cluster_std=1.0,
random_state=42)
X_anom = np.array([[6, 6], [7, -6], [-6, 7]])
X = np.vstack([X_normal, X_anom])
X = StandardScaler().fit_transform(X)

model = IsolationForest(contamination=0.01, random_state=42)
pred = model.fit_predict(X)      # -1 anomaly, 1 normal
scores = -model.score_samples(X) # larger means more anomalous

print("Anomaly indices:", np.where(pred == -1)[0])
print("Top scores:", np.sort(scores)[-5:])
```

8. Code Explanation

The example creates mostly normal data and a few extreme points. Isolation Forest learns random partitions. `fit_predict` returns anomaly labels, while `score_samples` gives continuous scores for ranking.

9. Training / Evaluation

If labels exist, use precision, recall, F1, PR-AUC, ROC-AUC, and cost-based evaluation. For unlabeled data, use alert volume, expert review, stability, and downstream incident detection.

Avoid random train/test splits for time-series anomaly detection. Use chronological validation.

10. Complexity and Cost

Cost depends on method. Statistical thresholds are cheap. Isolation Forest is efficient. Autoencoders require neural training and may need GPU for large data. Inference is usually cheap enough for real-time scoring.

11. Common Use Cases

- Credit card fraud
- Network intrusion
- Equipment failure prediction
- Medical abnormality screening
- Data quality monitoring
- ML drift detection

12. Common Mistakes

- Using accuracy on imbalanced anomaly data

- Training on heavily contaminated data
- Ignoring temporal leakage
- Setting threshold without business cost
- Not investigating false positives
- Assuming anomalies are always errors
- Forgetting feature scaling for distance methods

13. Edge Cases / Limitations

Anomaly detection is hard when anomalies resemble normal points, normal behavior changes over time, labels are missing, or anomalies are adversarial.

14. Variations

Variation	What Changes	When To Use	Importance
Statistical z-score/IQR	Simple thresholds	Univariate monitoring	High
Isolation Forest	Random isolation trees	Tabular anomalies	High
One-class SVM	Boundary around normal data	Small/medium data	High
Autoencoder	Reconstruction error	High-dimensional data	High
LOF	Local density comparison	Local anomalies	Medium

15. Related Topics

Isolation Forest and One-class SVM are common unsupervised anomaly algorithms. Autoencoders are useful for images, logs, and high-dimensional signals. DBSCAN noise points can also be used as anomalies.

16. Interview Questions

1. What is anomaly detection? Finding rare points that deviate from normal behavior.
2. Why is accuracy bad? A model predicting all normal can have high accuracy.
3. What metrics are better? Precision, recall, F1, PR-AUC, alert cost.
4. What is contamination? Expected anomaly fraction.
5. What is a contextual anomaly? A point abnormal only under context.
6. How choose threshold? Labels, business cost, alert capacity, contamination.
7. What if no labels exist? Use expert review and stability checks.
8. Why can train/test split be tricky? Time leakage can inflate performance.
9. How handle concept drift? Monitor score distributions and retrain.
10. Which model for tabular anomaly baseline? Isolation Forest.

17. Practice Tasks

- Build z-score anomaly detection for one feature.
- Compare Isolation Forest, LOF, and One-class SVM.
- Tune threshold for best F1.
- Simulate drift and observe false positives.

- Build alert ranking instead of only labels.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Fraud Alert Ranker	Ranks suspicious transactions	sklearn, pandas	Kaggle credit card fraud	Industry relevance
Sensor Fault Detector	Detects machine failures	sklearn, matplotlib	NASA turbofan	Predictive maintenance
Log Anomaly Monitor	Flags unusual app logs	sklearn, FastAPI	HDFS logs	MLOps/security

19. Quick Revision

- Key idea: assign high scores to rare/unusual points.
- Main formula: $\text{flag}(x) = s(x) > \tau$.
- When to use: rare event detection.
- Metrics: precision, recall, PR-AUC.
- Common traps: accuracy, leakage, bad thresholds.
- Interview one-liner: anomaly detection learns normal behavior and flags deviations.

20. Final Cheat Sheet

Item	Summary
Definition	Detect rare abnormal observations
Input/output	features -> anomaly score/flag
Main steps	preprocess, score, threshold, review
Hyperparameters	contamination, threshold, model-specific
Metrics	precision, recall, PR-AUC
Pros	useful without many labels
Cons	hard thresholding, false positives
Best use	fraud, monitoring, failures

Gaussian Mixture Models

1. Overview

Gaussian Mixture Models, or GMMs, are probabilistic clustering models that represent data as a mixture of multiple Gaussian distributions. Unlike K-Means, GMMs provide soft cluster membership probabilities.

2. Intuition

Instead of saying each point belongs fully to one cluster, GMM says a point may be 70% likely from one Gaussian and 30% from another. This is useful when clusters overlap.

3. Prerequisites

- Gaussian distribution
- Probability density
- Maximum likelihood
- Expectation-Maximization
- Covariance matrices

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Mixture component	One Gaussian in the model	Represents a cluster	Segment A	Soft clustering
Mixing weight	Prior probability of component	Cluster size	40% component 1	Must sum to 1
Mean	Component center	Location	Average profile	Like centroid
Covariance	Component shape/spread	Elliptical clusters	Correlated features	More flexible than K-Means
Responsibility	Probability point belongs to component	Soft label	$\gamma_{ik}=0.8$	E-step

5. Algorithm / Working Process

Input: X , number of components K .

Steps:

1. Initialize means, covariances, and weights.
2. E-step: compute responsibilities for each point-component pair.
3. M-step: update parameters using responsibilities.
4. Repeat until log-likelihood converges.
5. Output probabilities, labels, and density estimates.

Inference computes component probabilities for new points.

6. Mathematical Foundation

Mixture density:

$$p(x) = \sum_{k=1}^K \pi_k N(x \mid \mu_k, \Sigma_k)$$

Responsibility:

$$\gamma_{ik} = \pi_k N(x_i \mid \mu_k, \Sigma_k) / \sum_j \pi_j N(x_i \mid \mu_j, \Sigma_j)$$

Mean update:

$$\mu_k = \sum_i \gamma_{ik} x_i / \sum_i \gamma_{ik}$$

Log-likelihood:

$$L = \sum_i \log(\sum_k \pi_k N(x_i \mid \mu_k, \Sigma_k))$$

7. Practical Implementation

```
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.mixture import GaussianMixture

X, _ = make_blobs(n_samples=500, centers=3, cluster_std=1.5, random_state=42)
X = StandardScaler().fit_transform(X)

gmm = GaussianMixture(n_components=3, covariance_type="full", random_state=42)
gmm.fit(X)

labels = gmm.predict(X)
probs = gmm.predict_proba(X)
log_density = gmm.score_samples(X)

print("Means:\n", gmm.means_)
print("First probabilities:\n", probs[:3])
print("BIC:", gmm.bic(X))
```

8. Code Explanation

`GaussianMixture` fits Gaussian components using EM. `predict` gives the most likely component. `predict_proba` gives soft assignments. `score_samples` returns log-density, useful for anomaly detection.

9. Training / Evaluation

Evaluate with log-likelihood, BIC, AIC, silhouette on hard labels, and domain validation. Select `n_components` using BIC/AIC and business interpretability.

10. Complexity and Cost

Full covariance GMMs can be expensive in high dimensions because covariance matrices are $d \times d$. Each EM iteration costs roughly $O(n K d^2)$ for full covariance. Diagonal covariance reduces cost.

11. Common Use Cases

- Soft customer segmentation
- Density estimation
- Anomaly detection
- Speaker modeling
- Image segmentation
- Probabilistic clustering

12. Common Mistakes

- Forgetting scaling
- Using too many components
- Ignoring covariance type
- Assuming Gaussian clusters when data is not Gaussian
- Not checking convergence
- Confusing probability density with probability mass

13. Edge Cases / Limitations

GMMs struggle with non-Gaussian shapes, high dimensions, singular covariance matrices, and strong outliers. EM can converge to local optima.

14. Variations

Variation	What Changes	When To Use	Importance
Full covariance	Arbitrary ellipses	Flexible clusters	High
Diagonal covariance	Independent features per component	High-dimensional data	High
Tied covariance	Shared covariance	Small data	Medium
Bayesian GMM	Prior over components	Unknown cluster count	Medium

15. Related Topics

GMM generalizes K-Means: K-Means is like a limiting case with equal spherical covariance and hard assignments. GMMs also connect to anomaly detection through low-density scoring.

16. Interview Questions

1. What is a GMM? A mixture of Gaussian distributions.

2. What is soft clustering? Assigning probabilities instead of hard labels.
3. What algorithm trains GMMs? Expectation-Maximization.
4. What happens in E-step? Compute responsibilities.
5. What happens in M-step? Update means, covariances, and weights.
6. GMM vs K-Means? GMM is probabilistic and handles elliptical overlap.
7. How choose components? BIC, AIC, validation, domain sense.
8. What is covariance type? Constraint on component covariance matrices.
9. Can GMM detect anomalies? Yes, low likelihood points are suspicious.
10. Main weakness? Gaussian assumption and local optima.

17. Practice Tasks

- Fit GMM to blobs and plot probability contours.
- Compare covariance types.
- Use BIC to select K .
- Detect anomalies using low log-density.
- Compare GMM and K-Means on overlapping clusters.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Soft Segmenter	Gives customer membership probabilities	sklearn, pandas	Mall customers	Business interpretability
Density Anomaly Detector	Flags low-density records	sklearn	Credit card fraud	Risk modeling
Image Color Segmenter	Probabilistic pixel segmentation	OpenCV, sklearn	Images	CV fundamentals

19. Quick Revision

- Key idea: data comes from a mixture of Gaussians.
- Main formula: $p(\mathbf{x}) = \sum \pi_k N(\mathbf{x} | \mu_k, \Sigma_k)$.
- When to use: overlapping elliptical clusters.
- Metrics: log-likelihood, BIC, AIC.
- Common traps: too many components, bad covariance choice.
- Interview one-liner: GMM is soft probabilistic clustering trained by EM.

20. Final Cheat Sheet

Item	Summary
Definition	Probabilistic mixture of Gaussians
Input/output	$\mathbf{X}$ -> component probabilities
Main steps	E-step, M-step, repeat
Hyperparameters	components, covariance type

Item	Summary
Metrics	log-likelihood, BIC, AIC
Pros	soft labels, elliptical clusters
Cons	Gaussian assumption, expensive covariance
Best use	soft clustering and density estimation

Association Rule Mining

1. Overview

Association Rule Mining discovers relationships of the form $A \rightarrow B$ in transactional data. It answers questions like: "Customers who buy bread and butter also often buy milk."

It is used in recommendation systems, retail basket analysis, cross-selling, web usage mining, and medical co-occurrence analysis.

2. Intuition

If many shopping baskets containing **diapers** also contain **beer**, the store can use that association for promotions or shelf placement. The rule does not prove causation; it only shows co-occurrence.

3. Prerequisites

- Sets and transactions
- Conditional probability
- Basic counting
- pandas
- Evaluation metrics: support, confidence, lift

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Itemset	Set of items	Basis of rules	$\{\text{bread, milk}\}$	Frequent itemsets
Support	Frequency in all transactions	Filters rare patterns	20% baskets have milk	Anti-monotonic property
Confidence	$P(B A)$	$P(A B)$	Rule reliability	70% buy milk after bread
Lift	Confidence vs base rate	Measures useful association	$\text{lift} > 1$	Better than confidence alone
Rule	$A \rightarrow B$	Actionable pattern	$\{\text{chips}\} \rightarrow \{\text{salsa}\}$	Interpretability

5. Algorithm / Working Process

Input: transaction-item matrix.

Steps:

1. Convert transactions to one-hot encoded item matrix.
2. Find frequent itemsets above minimum support.
3. Generate candidate rules from itemsets.
4. Compute confidence, lift, leverage, and conviction.
5. Filter rules by thresholds.
6. Interpret rules with domain context.

6. Mathematical Foundation

Support:

$$\text{support}(A) = \text{count}(A) / N$$

Confidence:

$$\text{confidence}(A \rightarrow B) = \text{support}(A \cup B) / \text{support}(A)$$

Lift:

$$\text{lift}(A \rightarrow B) = \text{confidence}(A \rightarrow B) / \text{support}(B)$$

Lift greater than 1 means **A** and **B** occur together more than expected under independence.

7. Practical Implementation

```
# pip install mlxtend
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

transactions = [
    ["bread", "milk"],
    ["bread", "diaper", "beer", "egg"],
    ["milk", "diaper", "beer", "cola"],
    ["bread", "milk", "diaper", "beer"],
    ["bread", "milk", "diaper", "cola"],
]

encoder = TransactionEncoder()
```

```
encoded = encoder.fit(transactions).transform(transactions)
df = pd.DataFrame(encoded, columns=encoder.columns_)

itemsets = apriori(df, min_support=0.4, use_colnames=True)
rules = association_rules(itemsets, metric="lift", min_threshold=1.0)

print(itemsets)
print(rules[["antecedents", "consequents", "support", "confidence", "lift"]])
```

8. Code Explanation

`TransactionEncoder` converts lists of items into a boolean matrix. `apriori` finds frequent itemsets. `association_rules` creates rules and computes metrics such as support, confidence, and lift.

9. Training / Evaluation

There is no model training in the usual ML sense. Evaluation is by rule metrics and business validation. Useful metrics include support, confidence, lift, leverage, conviction, coverage, and actual A/B test impact.

10. Complexity and Cost

Candidate itemsets can grow exponentially with item count. Apriori reduces search using the principle that all subsets of a frequent itemset must also be frequent.

11. Common Use Cases

- Market basket analysis
- Product bundling
- Recommendation rules
- Medical symptom co-occurrence
- Web clickstream patterns
- Cross-sell campaigns

12. Common Mistakes

- Trusting confidence without lift
- Keeping extremely rare rules
- Confusing correlation with causation
- Ignoring seasonality
- Not removing duplicate transactions/items
- Producing too many rules without ranking

13. Edge Cases / Limitations

Association rules can explode combinatorially. They do not handle quantities, sequence, causality, or user personalization unless extended.

14. Variations

Variation	What Changes	When To Use	Importance
Apriori	Candidate generation by support	Small/medium baskets	High
FP-Growth	Frequent pattern tree	Larger transactions	High
Eclat	Vertical itemset format	Dense data	Medium
Sequential rules	Order-aware rules	Clickstream/time data	Medium

15. Related Topics

Association rules relate to recommender systems but are simpler and less personalized than matrix factorization. Apriori is a specific algorithm for frequent itemset mining.

16. Interview Questions

1. What is association rule mining? Finding co-occurrence rules in transactions.
2. What is support? Fraction of transactions containing an itemset.
3. What is confidence? Probability of consequent given antecedent.
4. What is lift? Confidence normalized by consequent frequency.
5. Why can confidence mislead? Popular consequents get high confidence anyway.
6. What does lift > 1 mean? Positive association beyond chance.
7. Is association rule mining supervised? No.
8. What is a frequent itemset? Itemset with support above threshold.
9. Why can rule mining be expensive? Candidate combinations grow exponentially.
10. Does a rule imply causation? No.

17. Practice Tasks

- Mine rules from grocery transactions.
- Compare rules ranked by confidence vs lift.
- Remove low-support rules and observe changes.
- Build a product bundle suggestion.
- Explain one misleading high-confidence rule.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Basket Rule Miner	Finds cross-sell product rules	pandas, mlxtend	Instacart/Groceries	Retail analytics
Medical Co-occurrence Explorer	Finds symptom-disease itemsets	pandas	MIMIC-style demo	Healthcare analytics
Web Path Rule Analyzer	Finds page co-visitation rules	pandas	Clickstream data	Product analytics

19. Quick Revision

- Key idea: discover $A \rightarrow B$ co-occurrence rules.
- Main formula: $\text{lift} = \text{confidence} / \text{support}(B)$.
- When to use: transactional baskets.
- Metrics: support, confidence, lift.
- Common traps: correlation vs causation, confidence-only ranking.
- Interview one-liner: association rules find interpretable co-occurrence patterns in transactions.

20. Final Cheat Sheet

Item	Summary
Definition	Rule mining from transaction data
Input/output	baskets -> itemsets/rules
Main steps	encode, frequent itemsets, generate rules
Hyperparameters	min support, min confidence/lift
Metrics	support, confidence, lift
Pros	interpretable, business-friendly
Cons	many rules, no causality
Best use	market basket analysis

Apriori Algorithm

1. Overview

Apriori is a classic algorithm for mining frequent itemsets. It powers association rule mining by finding item combinations that occur often enough in transaction data.

2. Intuition

If $\{\text{bread}, \text{milk}, \text{butter}\}$ is frequent, then $\{\text{bread}, \text{milk}\}$, $\{\text{bread}, \text{butter}\}$, and $\{\text{milk}, \text{butter}\}$ must also be frequent. Apriori uses this fact to avoid checking impossible large itemsets.

3. Prerequisites

- Set operations
- Transaction encoding
- Support metric
- Combinatorics
- Basic pandas

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Apriori property	Subsets of frequent itemset are frequent	Prunes search	Infrequent bread+jam blocks larger sets	Core trick
Candidate generation	Create possible k -itemsets	Search step	Join frequent pairs	Complexity
Pruning	Remove candidates with infrequent subsets	Saves work	Drop impossible triples	Efficiency
Support counting	Count candidate frequency	Decides frequent sets	Scan baskets	Bottleneck
Rule generation	Convert itemsets to rules	Business output	$\{A, B\} \rightarrow \{C\}$	Separate from itemset mining

5. Algorithm / Working Process

Input: transactions and minimum support.

Steps:

1. Find frequent 1-itemsets.
2. Generate candidate 2-itemsets from frequent 1-itemsets.
3. Count candidate support.
4. Keep itemsets meeting minimum support.
5. Repeat for larger k .
6. Stop when no candidates remain.
7. Generate association rules from frequent itemsets.

6. Mathematical Foundation

Apriori property:

If itemset I is frequent, every subset S subset I is frequent.
Contrapositive: if S is infrequent, every superset of S is infrequent.

Support:

$$\text{support}(I) = \text{number of transactions containing } I / \text{total transactions}$$

7. Practical Implementation

```
from itertools import combinations
```

```

transactions = [
    {"bread", "milk"},
    {"bread", "diaper", "beer", "egg"},
    {"milk", "diaper", "beer", "cola"},
    {"bread", "milk", "diaper", "beer"},
    {"bread", "milk", "diaper", "cola"},
]

def support(itemset):
    count = sum(itemset <= transaction for transaction in transactions)
    return count / len(transactions)

min_support = 0.4
items = sorted(set().union(*transactions))
frequent = []
current = [{item} for item in items if support({item}) >= min_support]

k = 1
while current:
    frequent.extend(current)
    candidates = []
    for a, b in combinations(current, 2):
        candidate = a | b
        if len(candidate) == k + 1 and candidate not in candidates:
            if all(set(sub) in current for sub in combinations(candidate, k)):
                candidates.append(candidate)
    current = [c for c in candidates if support(c) >= min_support]
    k += 1

for itemset in frequent:
    print(itemset, support(itemset))

```

8. Code Explanation

The code implements Apriori directly. It starts with frequent single items, joins them into larger candidates, prunes candidates whose subsets are not frequent, and keeps only itemsets above support threshold.

9. Training / Evaluation

Apriori has no train/test phase. Evaluate by support, number of discovered itemsets, rule quality after generation, interpretability, and business impact.

10. Complexity and Cost

Worst-case complexity is exponential in the number of items. Multiple database scans can be expensive. Apriori works best when minimum support is not too low and transactions are not extremely dense.

11. Common Use Cases

- Frequent product bundle discovery
- Feature co-occurrence mining

- Rule-based recommendations
- Medical code pattern mining
- Web event co-occurrence

12. Common Mistakes

- Setting minimum support too low
- Confusing frequent itemsets with association rules
- Not deduplicating items inside transactions
- Ignoring rare but important items
- Running Apriori on too many unique items
- Using it for ordered sequence patterns

13. Edge Cases / Limitations

Apriori is slow for dense datasets, low support thresholds, and large item vocabularies. It ignores order, quantity, user identity, and causality.

14. Variations

Variation	What Changes	When To Use	Importance
FP-Growth	Avoids candidate explosion with FP-tree	Large itemsets	High
Eclat	Uses vertical transaction IDs	Dense data	Medium
Hash-based Apriori	Hashes candidates	Speed improvement	Low
Dynamic Itemset Counting	Adds candidates during scans	Fewer passes	Low

15. Related Topics

Apriori is the frequent itemset mining step behind association rule mining. FP-Growth is a faster alternative. Matrix factorization solves recommendation through latent factors instead of explicit item rules.

16. Interview Questions

1. What is Apriori used for? Frequent itemset mining.
2. What is the Apriori property? All subsets of a frequent itemset are frequent.
3. Why is pruning valid? Infrequent subsets imply infrequent supersets.
4. What is minimum support? Frequency threshold for itemsets.
5. Apriori vs association rules? Apriori finds itemsets; rules are generated afterward.
6. Why can Apriori be slow? Candidate explosion and repeated scans.
7. What happens if support is too low? Too many candidates and noisy rules.
8. Does Apriori handle item order? No.
9. Alternative to Apriori? FP-Growth.
10. Is Apriori supervised? No.

17. Practice Tasks

- Implement support counting from scratch.
- Mine frequent itemsets from grocery data.
- Vary minimum support and plot itemset count.
- Generate rules from frequent triples.
- Compare Apriori with FP-Growth.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Frequent Basket Miner	Finds popular item bundles	Python, pandas	Instacart	Retail ML
Course Combo Analyzer	Finds courses students take together	pandas	University enrollment	EdTech analytics
Diagnosis Pattern Miner	Finds frequent symptom sets	pandas	Medical records demo	Healthcare analytics

19. Quick Revision

- Key idea: prune supersets of infrequent itemsets.
- Main formula: $\text{support}(I) = \text{count}(I) / N$.
- When to use: frequent itemset mining.
- Metrics: support and later rule metrics.
- Common traps: low support, huge item vocab.
- Interview one-liner: Apriori mines frequent itemsets using subset-based pruning.

20. Final Cheat Sheet

Item	Summary
Definition	Frequent itemset mining algorithm
Input/output	transactions -> frequent itemsets
Main steps	generate candidates, count support, prune
Hyperparameters	min support
Metrics	support, itemset count
Pros	simple, interpretable
Cons	exponential worst case
Best use	small/medium basket mining

Matrix Factorization

1. Overview

Matrix factorization decomposes a large matrix into smaller latent-factor matrices. In ML interviews it is most often discussed for recommender systems, where a user-item rating matrix is factorized into user and item embeddings.

2. Intuition

A movie rating matrix may be explained by hidden factors like action preference, romance preference, and comedy preference. Each user and movie gets coordinates in this hidden factor space. The dot product predicts ratings.

3. Prerequisites

- Linear algebra
- Dot product
- Gradient descent
- Regularization
- Sparse matrices
- Recommender metrics

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
User factors	User embedding vector	Captures preferences	User likes action	Latent features
Item factors	Item embedding vector	Captures item attributes	Movie is action-heavy	Dot-product prediction
Sparse matrix	Most ratings missing	Real recommender challenge	User rated 20 of 10k movies	Missing not equal zero
Reconstruction	Approximate original ratings	Prediction goal	$R \approx U V^T$	Low-rank assumption
Regularization	Penalizes large factors	Prevents overfitting	L2 penalty	Cold-start and sparsity

5. Algorithm / Working Process

Input: observed user-item interactions.

Steps:

1. Create sparse rating matrix R .
2. Initialize user matrix P and item matrix Q .
3. Predict rating with dot product.
4. Compute loss only on observed entries.
5. Update factors with SGD/ALS.

6. Recommend items with highest predicted scores.

6. Mathematical Foundation

Prediction:

$$\hat{r}_{ui} = \mathbf{p}_u^T \mathbf{q}_i$$

With biases:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^T \mathbf{q}_i$$

Objective:

$$\min \sum_{(u,i) \text{ in observed}} (r_{ui} - \mathbf{p}_u^T \mathbf{q}_i)^2 + \lambda (||\mathbf{p}_u||^2 + ||\mathbf{q}_i||^2)$$

Implicit feedback often uses confidence-weighted loss:

$$\min \sum_{u,i} c_{ui} (\hat{r}_{ui} - r_{ui})^2 + \lambda (||\mathbf{p}_u||^2 + ||\mathbf{q}_i||^2)$$

7. Practical Implementation

```
import numpy as np

ratings = np.array([
    [5, 4, 0, 0],
    [4, 0, 0, 1],
    [1, 0, 5, 4],
    [0, 1, 4, 5],
], dtype=float)

n_users, n_items = ratings.shape
k = 2
lr = 0.01
reg = 0.02
rng = np.random.default_rng(42)
P = rng.normal(0, 0.1, size=(n_users, k))
Q = rng.normal(0, 0.1, size=(n_items, k))

observed = np.argwhere(ratings > 0)

for _ in range(1000):
```

```
for u, i in observed:
    err = ratings[u, i] - P[u] @ Q[i]
    P[u] += lr * (err * Q[i] - reg * P[u])
    Q[i] += lr * (err * P[u] - reg * Q[i])

predicted = P @ Q.T
print(np.round(predicted, 2))
```

8. Code Explanation

Zeros represent missing ratings, not real zero ratings. The loop updates only observed entries. $P[u] @ Q[i]$ predicts a rating from user and item latent vectors.

9. Training / Evaluation

Use train/test split over observed interactions, not matrix cells randomly without care. Explicit ratings use RMSE/MAE. Ranking recommenders use Precision@K, Recall@K, MAP@K, NDCG@K, and hit rate.

10. Complexity and Cost

SGD cost per epoch is $O(|\text{observed}| * k)$. Memory is $O((\text{users} + \text{items}) * k)$. It scales well to sparse data. GPU helps for neural recommenders but is not required for basic factorization.

11. Common Use Cases

- Movie recommendation
- Product recommendation
- Music recommendation
- Collaborative filtering
- Embedding users/items
- Missing value approximation

12. Common Mistakes

- Treating missing entries as zero ratings
- Randomly splitting after leakage through users/items
- Evaluating ranking with RMSE only
- Ignoring cold-start users/items
- Not using regularization
- Recommending already-consumed items

13. Edge Cases / Limitations

Matrix factorization struggles with cold start, changing preferences, sparse users, popularity bias, and lack of side information unless extended.

14. Variations

Variation	What Changes	When To Use	Importance
SVD	Matrix decomposition	Dense/filled matrices	Medium
FunkSVD	SGD on observed ratings	Recommenders	High
ALS	Alternating least squares	Large sparse data	High
NMF	Non-negative factors	Interpretability	Medium
Neural CF	Neural interaction function	Complex recommenders	Medium

15. Related Topics

Matrix factorization relates to PCA because both are low-rank approximations. It differs from association rules because it learns latent preferences rather than explicit item co-occurrence rules.

16. Interview Questions

1. What is matrix factorization? Decomposing a matrix into lower-rank factor matrices.
2. How used in recommendations? Factor user-item matrix into user and item embeddings.
3. What is predicted rating? Dot product of user and item vectors.
4. Why not treat missing as zero? Missing means unknown, not dislike.
5. What is cold start? New users/items lack interactions.
6. How evaluate recommendations? Ranking metrics like Recall@K and NDCG@K.
7. What does regularization do? Prevents overfitting latent factors.
8. ALS vs SGD? ALS solves alternating least squares; SGD updates observed entries.
9. What are latent factors? Hidden dimensions explaining preferences.
10. How add side information? Hybrid models with content/user features.

17. Practice Tasks

- Implement SGD matrix factorization.
- Add user and item biases.
- Evaluate RMSE on held-out ratings.
- Build top-10 recommendation list.
- Compare with item-item collaborative filtering.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Movie Recommender	Predicts and ranks movies	NumPy, pandas	MovieLens	Classic ML project
Product Recommender	Suggests products from purchases	implicit/sklearn	Instacart	E-commerce relevance
Music Taste Embeddings	Finds user/song factors	Python	Last.fm	Recommendation depth

19. Quick Revision

- Key idea: approximate R as $P Q^T$.
- Main formula: $\hat{r}_{ui} = p_u^T q_i$.
- When to use: collaborative filtering.
- Metrics: RMSE, Recall@K, NDCG@K.
- Common traps: missing-as-zero, cold start.
- Interview one-liner: matrix factorization learns user and item embeddings from sparse interactions.

20. Final Cheat Sheet

Item	Summary
Definition	Low-rank decomposition
Input/output	user-item matrix -> latent factors
Main steps	initialize, predict, optimize observed loss
Hyperparameters	rank, learning rate, regularization
Metrics	RMSE, Recall@K, NDCG@K
Pros	scalable, strong recommender baseline
Cons	cold start, sparsity
Best use	collaborative filtering

Autoencoders

1. Overview

Autoencoders are neural networks trained to reconstruct their input. They learn compressed latent representations and are used for dimensionality reduction, denoising, anomaly detection, representation learning, and generative modeling foundations.

2. Intuition

An autoencoder is like asking a student to summarize a paragraph and then reconstruct the original from the summary. If the summary is useful, it captures important structure.

3. Prerequisites

- Neural networks
- Backpropagation
- Loss functions
- PyTorch basics
- Train/validation split
- Regularization

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Encoder	Maps input to latent code	Compression	Image -> vector	Representation learning
Bottleneck	Low-dimensional latent layer	Forces compression	784 -> 32	Prevent identity copying
Decoder	Reconstructs input	Training target	vector -> image	Generative side
Reconstruction loss	Measures output vs input	Optimization objective	MSE/BCE	Anomaly scoring
Latent space	Learned representation	Useful features	Similar images nearby	Downstream tasks

5. Algorithm / Working Process

Input: data x .

Steps:

- 1. Encoder maps x to latent code z .
- 2. Decoder maps z to reconstruction $\hat{x}$.
- 3. Compute reconstruction loss.
- 4. Backpropagate loss.
- 5. Use encoder for features or reconstruction error for anomaly detection.

Inference returns reconstruction, latent embedding, or anomaly score.

6. Mathematical Foundation

Encoder and decoder:

```
z = f_theta(x)
x_hat = g_phi(z)
```

MSE reconstruction loss:

$$L = (1/n) \sum_i ||x_i - \hat{x}_i||^2$$

Anomaly score:

$$s(x) = ||x - \hat{x}||^2$$

Denoising autoencoder trains with corrupted input:

```
x_hat = g(f(noisy_x)), target = clean_x
```

7. Practical Implementation

```
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset
from sklearn.datasets import load_digits
from sklearn.preprocessing import MinMaxScaler

X, _ = load_digits(return_X_y=True)
X = MinMaxScaler().fit_transform(X).astype("float32")
loader = DataLoader(TensorDataset(torch.tensor(X)), batch_size=64, shuffle=True)

class Autoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(nn.Linear(64, 32), nn.ReLU(), nn.Linear(32,
8))
        self.decoder = nn.Sequential(nn.Linear(8, 32), nn.ReLU(), nn.Linear(32,
64), nn.Sigmoid())

    def forward(self, x):
        z = self.encoder(x)
        return self.decoder(z)

model = Autoencoder()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.MSELoss()

for epoch in range(20):
    total = 0.0
    for (batch,) in loader:
        recon = model(batch)
        loss = loss_fn(recon, batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total += loss.item() * len(batch)
    print(epoch, total / len(X))
```

8. Code Explanation

The encoder compresses 64 digit pixels to an 8-dimensional code. The decoder reconstructs the original input. MSE trains the model to preserve important information through the bottleneck.

9. Training / Evaluation

Use train/validation split. Evaluate reconstruction loss, downstream feature quality, anomaly detection PR-AUC, or visual reconstruction quality. Avoid training anomaly detectors on many anomalies.

Hyperparameters include latent dimension, architecture, learning rate, batch size, epochs, regularization, and loss function.

10. Complexity and Cost

Cost depends on network size and data. For tabular data, CPU can be enough. For images/audio, GPU is useful. Inference is one forward pass.

11. Common Use Cases

- Dimensionality reduction
- Denoising
- Anomaly detection
- Feature learning
- Image reconstruction
- Pretraining representations

12. Common Mistakes

- Making bottleneck too large
- Evaluating only training reconstruction loss
- Training anomaly autoencoder on contaminated data
- Forgetting normalization
- Using output activation incompatible with data range
- Assuming good reconstruction means useful features

13. Edge Cases / Limitations

Autoencoders can learn identity mappings, reconstruct anomalies too well, overfit small data, and be less interpretable than PCA.

14. Variations

Variation	What Changes	When To Use	Importance
Denoising AE	Corrupt input, reconstruct clean	Noise removal	High
Sparse AE	Adds sparsity penalty	Interpretable features	Medium
Variational AE	Probabilistic latent distribution	Generative modeling	High
Convolutional AE	CNN encoder/decoder	Images	High
Sequence AE	RNN/Transformer encoder	Time series/text	Medium

15. Related Topics

Autoencoders relate to PCA as nonlinear dimensionality reduction. VAEs connect autoencoders to generative AI. Reconstruction error connects them to anomaly detection.

16. Interview Questions

1. What is an autoencoder? A neural network trained to reconstruct its input.
2. Why use a bottleneck? To force compressed representation learning.
3. What is reconstruction loss? Difference between input and output.
4. Autoencoder vs PCA? Autoencoder can learn nonlinear mappings; PCA is linear.
5. How detect anomalies? High reconstruction error indicates abnormal input.
6. What if latent dimension is too large? Model may copy input.
7. What is denoising autoencoder? Reconstructs clean input from noisy input.
8. What is VAE? Probabilistic autoencoder with latent distribution regularization.
9. Which loss for normalized images? MSE or BCE depending on modeling choice.
10. How prevent overfitting? Smaller bottleneck, regularization, validation, dropout.

17. Practice Tasks

- Train an autoencoder on digits.
- Plot original vs reconstructed images.
- Use reconstruction error for anomaly detection.
- Compare PCA and autoencoder compression.
- Add noise and train a denoising autoencoder.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Anomaly Autoencoder	Flags abnormal records	PyTorch, sklearn	Credit card fraud	Deep anomaly detection
Image Denoiser	Removes noise from images	PyTorch, torchvision	MNIST/CIFAR	Computer vision
Latent Explorer	Visualizes learned codes	PyTorch, UMAP	Fashion-MNIST	Representation learning

19. Quick Revision

- Key idea: reconstruct input through compressed latent code.
- Main formula: $L = ||x - x_{\text{hat}}||^2$.
- When to use: compression, denoising, anomaly detection.
- Metrics: reconstruction loss, PR-AUC for anomalies.
- Common traps: oversized bottleneck, contamination.
- Interview one-liner: autoencoders learn compact representations by reconstructing inputs.

20. Final Cheat Sheet

Item	Summary
Definition	Neural reconstruction model
Input/output	$x \rightarrow \hat{x}$ and latent z
Main steps	encode, decode, minimize reconstruction loss
Hyperparameters	latent size, layers, lr, epochs
Metrics	reconstruction loss, downstream score
Pros	nonlinear, flexible
Cons	needs tuning, less interpretable
Best use	compression/anomaly/denoising

Self-Organizing Maps

1. Overview

Self-Organizing Maps, or SOMs, are neural unsupervised models that project high-dimensional data onto a usually 2D grid while preserving topology. Nearby grid cells represent similar input patterns.

2. Intuition

Imagine a 2D map whose cells compete to represent input points. When one cell wins, it and its neighbors move closer to the input. Over time, nearby cells specialize in similar data.

3. Prerequisites

- Vectors and distances
- Competitive learning
- Learning rate schedules
- Clustering intuition
- Visualization

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Grid neuron	Cell with weight vector	Represents prototype	10x10 map cell	Prototype learning
Best Matching Unit	Closest neuron to input	Winner for update	BMU for customer vector	Competitive learning
Neighborhood	Nearby cells around BMU	Preserves topology	Update adjacent cells	Smooth map

Concept	Meaning	Why It Matters	Example	Interview Angle
Learning rate	Update strength	Controls convergence	Decays over epochs	Stability
U-Matrix	Distance visualization	Shows cluster boundaries	Bright gaps between clusters	Interpretability

5. Algorithm / Working Process

Input: numeric vectors and grid size.

Steps:

1. Initialize grid neuron weights.
2. Pick an input sample.
3. Find Best Matching Unit by distance.
4. Update BMU and neighboring neurons toward the input.
5. Decay learning rate and neighborhood radius.
6. Repeat for many iterations.

Output is a 2D topology-preserving map.

6. Mathematical Foundation

BMU:

$$c = \operatorname{argmin}_j ||x - w_j||$$

Weight update:

$$w_j(t+1) = w_j(t) + \alpha(t) h_{cj}(t) (x - w_j(t))$$

Neighborhood function:

$$h_{cj}(t) = \exp(-||r_c - r_j||^2 / (2 \sigma(t)^2))$$

7. Practical Implementation

```
# pip install minisom
from sklearn.datasets import load_iris
from sklearn.preprocessing import MinMaxScaler
from minisom import MiniSom

X, y = load_iris(return_X_y=True)
```

```
X = MinMaxScaler().fit_transform(X)

som = MiniSom(x=8, y=8, input_len=X.shape[1], sigma=1.0, learning_rate=0.5,
random_seed=42)
som.random_weights_init(X)
som.train_random(X, num_iteration=1000)

for sample, label in zip(X[:5], y[:5]):
    print("BMU:", som.winner(sample), "label:", label)
```

8. Code Explanation

MiniSom creates an 8x8 neuron grid. Each neuron has a weight vector matching input dimension. Training repeatedly finds the closest neuron and updates it plus neighbors.

9. Training / Evaluation

Evaluate with quantization error, topographic error, visualization quality, and downstream cluster usefulness. Tune grid size, learning rate, sigma, and iterations.

10. Complexity and Cost

Each update checks all grid neurons, so cost is approximately $O(\text{iterations} * \text{grid_size} * d)$. SOMs are usually CPU-friendly for small and medium datasets.

11. Common Use Cases

- High-dimensional visualization
- Customer segmentation
- Pattern discovery
- Gene expression maps
- Industrial process monitoring

12. Common Mistakes

- Using SOM without scaling
- Choosing huge grids for small data
- Reading grid coordinates as exact distances
- Not training long enough
- Ignoring random initialization
- Treating SOM as a modern default over UMAP/t-SNE

13. Edge Cases / Limitations

SOMs are less common in modern production, sensitive to hyperparameters, and can be harder to evaluate objectively.

14. Variations

Variation	What Changes	When To Use	Importance
Growing SOM	Grid expands	Unknown map size	Low/Research
Batch SOM	Batch updates	Stable training	Medium
Supervised SOM	Uses labels	Classification visualization	Low
Hierarchical SOM	Multiple SOM layers	Multi-level maps	Low

15. Related Topics

SOMs relate to K-Means because both learn prototypes. SOMs add neighborhood structure. UMAP and t-SNE are more common modern visualization alternatives.

16. Interview Questions

1. What is a SOM? A topology-preserving neural map for unsupervised learning.
2. What is BMU? The neuron whose weight is closest to the input.
3. Why update neighbors? To preserve topology.
4. What does sigma control? Neighborhood radius.
5. What is quantization error? Average distance from points to BMUs.
6. SOM vs K-Means? SOM has a grid and neighborhood updates.
7. Why scale data? BMU selection uses distance.
8. Is SOM supervised? Standard SOM is unsupervised.
9. Main use today? Visualization and exploratory clustering.
10. Main weakness? Hyperparameter sensitivity and limited modern adoption.

17. Practice Tasks

- Train SOM on Iris.
- Visualize U-Matrix.
- Compare SOM clusters with K-Means.
- Tune grid size.
- Use SOM for anomaly detection via BMU distance.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Customer SOM Map	Maps customer profiles to grid	MiniSom, pandas	Mall customers	Visual segmentation
Sensor State Map	Shows machine operating states	MiniSom	Sensor data	Industrial AI
Gene Pattern Map	Visualizes gene expression	MiniSom, seaborn	Gene data	Research analytics

19. Quick Revision

- Key idea: competitive prototype grid preserving topology.
- Main formula: $w \leftarrow w + \alpha h (x-w)$.
- When to use: visual pattern discovery.
- Metrics: quantization/topographic error.
- Common traps: no scaling, overinterpreting map.
- Interview one-liner: SOM maps high-dimensional data to a topology-preserving 2D grid.

20. Final Cheat Sheet

Item	Summary
Definition	Competitive neural clustering map
Input/output	vectors -> grid BMUs
Main steps	find BMU, update BMU and neighbors
Hyperparameters	grid size, sigma, learning rate
Metrics	quantization error, topographic error
Pros	visual and interpretable
Cons	old, parameter-sensitive
Best use	exploratory maps

Spectral Clustering

1. Overview

Spectral clustering uses graph theory and eigenvectors to cluster data. It is powerful when clusters are non-convex or better represented by connectivity than by centroid distance.

2. Intuition

Build a graph where similar points are connected. Then cut the graph so strong connections stay inside clusters and weak connections go between clusters. Eigenvectors help reveal this graph structure.

3. Prerequisites

- Graphs and adjacency matrices
- Eigenvectors/eigenvalues
- Similarity kernels
- K-Means basics
- Linear algebra

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Similarity graph	Weighted graph of points	Converts data to graph	RBF similarity	Graph construction
Adjacency matrix	Similarity weights	Input to Laplacian	W_{ij}	Sparse vs dense
Degree matrix	Sum of edge weights	Normalization	$D_{ii} = \sum_j W_{ij}$	Laplacian
Graph Laplacian	Matrix encoding graph cuts	Spectral embedding	$L = D - W$	Eigenvectors
Spectral embedding	Eigenvector representation	Makes clusters separable	K-Means on eigenvectors	Core pipeline

5. Algorithm / Working Process

Input: data X , cluster count K , affinity method.

Steps:

- 1. Build similarity graph.
- 2. Compute graph Laplacian.
- 3. Compute first K eigenvectors.
- 4. Treat rows of eigenvector matrix as new features.
- 5. Run K-Means on these spectral features.
- 6. Output cluster labels.

6. Mathematical Foundation

Adjacency matrix:

$$W_{ij} = \exp(-||x_i - x_j||^2 / (2 \sigma^2))$$

Degree matrix:

$$D_{ii} = \sum_j W_{ij}$$

Unnormalized Laplacian:

$$L = D - W$$

Normalized Laplacian:

$$L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}$$

Spectral clustering approximates normalized graph cut objectives.

7. Practical Implementation

```
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import SpectralClustering
from sklearn.metrics import adjusted_rand_score

X, y_true = make_moons(n_samples=400, noise=0.06, random_state=42)
X = StandardScaler().fit_transform(X)

model = SpectralClustering(
    n_clusters=2,
    affinity="nearest_neighbors",
    n_neighbors=10,
    assign_labels="kmeans",
    random_state=42
)
labels = model.fit_predict(X)

print("ARI:", adjusted_rand_score(y_true, labels))
```

8. Code Explanation

`affinity="nearest_neighbors"` builds a sparse graph where each point connects to nearby points. Spectral clustering embeds this graph using eigenvectors, then uses K-Means for final labels.

9. Training / Evaluation

If labels exist, use ARI or NMI. Without labels, use silhouette carefully because clusters may be non-convex. Tune `n_clusters`, affinity, gamma, and neighbor count.

10. Complexity and Cost

Eigen decomposition can be expensive, often near $O(n^3)$ for dense matrices. Sparse nearest-neighbor graphs help. It is usually for small/medium datasets, not millions of points.

11. Common Use Cases

- Non-convex clustering
- Image segmentation
- Graph/community detection
- Manifold data clustering
- Similarity-based clustering

12. Common Mistakes

- Using dense affinity for large data
- Bad gamma or neighbor count
- Forgetting final K-Means step conceptually
- Expecting scalability like K-Means
- Using poor similarity metric
- Treating silhouette as always reliable

13. Edge Cases / Limitations

Spectral clustering is expensive, sensitive to graph construction, and needs K . It can fail if the similarity graph disconnects badly or connects clusters too strongly.

14. Variations

Variation	What Changes	When To Use	Importance
Normalized spectral clustering	Uses normalized Laplacian	Unequal degrees	High
RBF affinity	Fully connected weighted graph	Small smooth data	Medium
kNN affinity	Sparse local graph	Larger/manifold data	High
Nyström approximation	Approximate eigenvectors	Larger data	Medium

15. Related Topics

Spectral clustering relates to graph cuts, Laplacian eigenmaps, manifold learning, and K-Means. It can solve shapes where K-Means fails but costs more.

16. Interview Questions

1. What is spectral clustering? Clustering using eigenvectors of a graph Laplacian.
2. Why use a graph? It captures connectivity and similarity.
3. What is graph Laplacian? $L = D - W$.
4. Why run K-Means at the end? Eigenvectors produce embedding rows that need partitioning.
5. When prefer it over K-Means? Non-convex or graph-structured clusters.
6. What is affinity? Similarity between points.
7. Main bottleneck? Eigen decomposition.
8. Does it need K ? Yes.
9. What if graph is poorly built? Clusters are poor.
10. Spectral clustering vs DBSCAN? Spectral uses graph cuts and needs K ; DBSCAN uses density and detects noise.

17. Practice Tasks

- Cluster two-moons data.
- Compare RBF and nearest-neighbor affinity.
- Vary `n_neighbors`.

- Implement Laplacian for a tiny graph.
- Compare with K-Means and DBSCAN.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Image Segmenter	Segments image regions by similarity	sklearn, OpenCV	Berkeley images	CV + graph ML
Community Detector	Finds groups in similarity graph	networkx, sklearn	social graph	Graph ML
Shape Cluster Lab	Demonstrates non-convex clustering	sklearn, streamlit	synthetic shapes	Interview demo

19. Quick Revision

- Key idea: cluster eigenvector embedding of similarity graph.
- Main formula: $L = D - W$.
- When to use: non-convex graph-like clusters.
- Metrics: ARI/NMI, graph inspection.
- Common traps: bad affinity and high cost.
- Interview one-liner: spectral clustering turns clustering into a graph partitioning problem.

20. Final Cheat Sheet

Item	Summary
Definition	Graph eigenvector clustering
Input/output	data/similarity graph -> labels
Main steps	graph, Laplacian, eigenvectors, K-Means
Hyperparameters	clusters, affinity, gamma, neighbors
Metrics	ARI, NMI, visual checks
Pros	handles non-convex clusters
Cons	expensive, sensitive graph
Best use	graph/manifold clustering

Isolation Forest Deeply

1. Overview

Isolation Forest is an unsupervised anomaly detection algorithm based on the idea that anomalies are easier to isolate than normal points. It builds random trees and scores points by average path length.

It is a top interview algorithm for tabular anomaly detection because it is fast, scalable, and works without labels.

2. Intuition

In a dataset of normal transactions, most points are packed in dense regions. Random splits need many cuts to isolate one normal point. An outlier sitting far away can be isolated with very few random cuts.

3. Prerequisites

- Decision trees
- Random sampling
- Path length
- Anomaly detection metrics
- Feature preprocessing

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Isolation	Separating a point from others	Core anomaly idea	Far point split quickly	Different from density
Random split	Random feature and threshold	Creates isolation tree	Split age at 43.2	No supervised impurity
Path length	Splits needed to isolate point	Anomaly signal	Short path = anomaly	Main scoring idea
Subsampling	Train each tree on sample	Improves speed and anomaly contrast	256 samples/tree	Scalability
Contamination	Expected anomaly proportion	Threshold labels	1% alerts	Score vs decision

5. Algorithm / Working Process

Input: feature matrix X .

Training:

1. For each tree, sample a subset of rows.
2. Randomly select a feature.
3. Randomly choose a split between min and max feature value.
4. Recursively split until point is isolated, max depth is reached, or node has one value.
5. Repeat for many trees.

Inference:

1. Pass point through all trees.
2. Compute average path length.
3. Convert path length to anomaly score.
4. Threshold score if labels are needed.

6. Mathematical Foundation

Average path length:

$$E(h(x)) = \text{average path length of } x \text{ across trees}$$

Normalization constant for sample size ψ :

$$c(\psi) = 2 H(\psi - 1) - 2(\psi - 1)/\psi$$

where $H(i)$ is the harmonic number.

Anomaly score:

$$s(x, \psi) = 2^{-E(h(x)) / c(\psi)}$$

Interpretation:

- s near 1: likely anomaly
- s around 0.5: uncertain
- s much less than 0.5: likely normal

7. Practical Implementation

```
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.metrics import classification_report, average_precision_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(42)
normal = rng.normal(0, 1, size=(1000, 4))
anomaly = rng.normal(5, 1, size=(40, 4))
X = np.vstack([normal, anomaly])
y = np.array([0] * len(normal) + [1] * len(anomaly))

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = IsolationForest(
    n_estimators=200,
    max_samples=256,
    contamination=0.04,
    random_state=42
)
model.fit(X_train)

scores = -model.score_samples(X_test)
pred = (model.predict(X_test) == -1).astype(int)

print("PR-AUC:", average_precision_score(y_test, scores))
print(classification_report(y_test, pred))
```

8. Code Explanation

The training set contains mostly normal data. Scaling is used because mixed feature scales can affect random threshold behavior. `score_samples` is negated so larger values mean more anomalous. PR-AUC is better than accuracy for rare anomalies.

9. Training / Evaluation

Use labels if available only for evaluation/thresholding, not model fitting. Metrics: precision, recall, F1, PR-AUC, ROC-AUC, alert rate, and investigation cost.

Important hyperparameters:

Hyperparameter	Meaning	Practical Tip
<code>n_estimators</code>	Number of trees	More trees improve stability
<code>max_samples</code>	Subsample size per tree	256 is common default
<code>contamination</code>	Expected anomaly ratio	Sets decision threshold
<code>max_features</code>	Feature subsampling	Useful with many noisy features

10. Complexity and Cost

Training is roughly $O(t * \text{psi} * \log \text{psi})$ where `t` is trees and `psi` is subsample size. Inference is $O(t * \text{tree_depth})$. It is CPU-friendly and scales well.

11. Common Use Cases

- Fraud detection baseline
- Cybersecurity event detection
- Data quality monitoring

- Sensor fault detection
- Outlier filtering before modeling
- Transaction alert ranking

12. Common Mistakes

- Evaluating with accuracy
- Using labels during unsupervised fitting incorrectly
- Setting contamination equal to a guess without validating alert volume
- Forgetting time-based validation
- Not checking feature drift
- Assuming all rare points are bad
- Ignoring categorical encoding quality

13. Edge Cases / Limitations

Isolation Forest struggles when anomalies are not easier to isolate, when anomalies form dense groups, when features are irrelevant/noisy, or when normal data is highly multi-modal.

14. Variations

Variation	What Changes	When To Use	Importance
Extended Isolation Forest	Random hyperplane splits	Axis-aligned splits fail	Medium
SCiForest	Uses selected split criteria	Better anomaly separation	Low
Streaming variants	Updates over time	Real-time monitoring	Medium
Feature bagging IF	Subsample features	High-dimensional data	Medium

15. Related Topics

Isolation Forest is related to Random Forest only structurally; it does not use labels or impurity. It compares with One-class SVM as a faster tabular baseline. It compares with LOF because LOF uses local density while Isolation Forest uses random partition depth.

16. Interview Questions

1. Why do anomalies have shorter paths? They are rare and different, so random splits isolate them quickly.
2. Is Isolation Forest supervised? No, labels are not required.
3. What does contamination do? Sets threshold for converting scores to anomaly labels.
4. What is `max_samples`? Number of samples used to build each tree.
5. Why subsample? It improves speed and makes anomalies stand out.
6. Does it use Gini or entropy? No, splits are random.
7. Which metric for rare anomalies? PR-AUC, precision, recall, and F1.
8. What does `predict` return in sklearn? `-1` for anomalies and `1` for normal.
9. When does it fail? Dense anomaly clusters or poor features.
10. Isolation Forest vs One-class SVM? Isolation Forest scales better and needs less kernel tuning.

17. Practice Tasks

- Build Isolation Forest on synthetic anomalies.
- Tune contamination for a target alert rate.
- Compare PR-AUC with ROC-AUC.
- Plot anomaly scores over time.
- Test effect of noisy irrelevant features.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Fraud Triage System	Ranks risky transactions	sklearn, FastAPI	Kaggle fraud	Production-style anomaly ML
Sensor Drift Monitor	Detects abnormal machine states	pandas, sklearn	NASA turbofan	Industrial AI
Data Quality Sentinel	Flags weird rows in pipelines	sklearn, Great Expectations	Any tabular data	MLOps relevance

19. Quick Revision

- Key idea: anomalies are isolated in fewer random splits.
- Main formula: $s(x) = 2^{(-E(h(x))/c(\psi))}$.
- When to use: tabular anomaly detection baseline.
- Metrics: PR-AUC, precision, recall.
- Common traps: accuracy and bad contamination.
- Interview one-liner: Isolation Forest detects anomalies by measuring how quickly random trees isolate each point.

20. Final Cheat Sheet

Item	Summary
Definition	Random-tree anomaly detector
Input/output	features -> anomaly score/label
Main steps	random trees, path length, score
Hyperparameters	trees, max samples, contamination
Metrics	PR-AUC, precision, recall
Pros	fast, scalable, no labels
Cons	weak if anomalies are not easily isolated
Best use	tabular anomaly baseline

One-Class SVM

1. Overview

One-class SVM is an unsupervised or semi-supervised anomaly detection method that learns a boundary around normal data. Points outside the boundary are considered anomalies.

It is useful for novelty detection when training data is mostly clean and normal.

2. Intuition

Imagine drawing a flexible fence around normal examples. Future points inside the fence are accepted as normal; points outside are rejected as anomalies.

3. Prerequisites

- SVM margin intuition
- Kernels
- Feature scaling
- Optimization basics
- Anomaly metrics

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Support vectors	Boundary-defining points	Define decision frontier	Edge normal samples	SVM connection
Kernel	Similarity function	Nonlinear boundary	RBF kernel	Gamma tuning
ν	Upper bound on outlier fraction, lower bound on support vectors	Controls strictness	$\nu=0.05$	Common question
Decision function	Signed distance to boundary	Anomaly score	Negative outside	Thresholding
Novelty detection	Train on normal data only	Clean setting	Manufacturing normal samples	Difference from outlier detection

5. Algorithm / Working Process

Input: mostly normal data.

Training:

1. Scale features.

2. Choose kernel, often RBF.
3. Learn boundary that separates data from origin in feature space.
4. Allow some violations controlled by ν .

Inference:

1. Compute decision score.
2. Positive score means normal.
3. Negative score means anomaly.

6. Mathematical Foundation

One-class SVM solves:

$$\min_{\{w, \rho, \xi\}} \frac{1}{2} \|w\|^2 + \frac{1}{(\nu n)} \sum_i \xi_i - \rho$$

subject to:

$$w \cdot \phi(x_i) \geq \rho - \xi_i$$

$$\xi_i \geq 0$$

Decision function:

$$f(x) = \text{sign}(w \cdot \phi(x) - \rho)$$

RBF kernel:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

7. Practical Implementation

```
import numpy as np
from sklearn.svm import OneClassSVM
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report

rng = np.random.default_rng(42)
X_train = rng.normal(0, 1, size=(400, 2)) # mostly normal
X_test_normal = rng.normal(0, 1, size=(100, 2))
X_test_anom = rng.normal(4, 1, size=(20, 2))
X_test = np.vstack([X_test_normal, X_test_anom])
y_test = np.array([0] * 100 + [1] * 20)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
model = OneClassSVM(kernel="rbf", gamma="scale", nu=0.05)
model.fit(X_train)

pred = (model.predict(X_test) == -1).astype(int)
scores = -model.decision_function(X_test)

print(classification_report(y_test, pred))
print("Top anomaly scores:", np.sort(scores.ravel())[-5:])
```

8. Code Explanation

The model trains only on normal-looking data. `nu=0.05` allows roughly 5% training violations. `predict` returns `-1` for anomalies. The negative decision function is used as an anomaly score where larger means more suspicious.

9. Training / Evaluation

Train on clean normal data when possible. Validate using labeled anomalies, simulated anomalies, or expert review. Metrics include precision, recall, F1, PR-AUC, false positive rate, and alert volume.

10. Complexity and Cost

Kernel One-class SVM can be expensive, often between $O(n^2)$ and $O(n^3)$ training depending on solver/data. It is not ideal for very large datasets. Linear variants scale better.

11. Common Use Cases

- Manufacturing defect detection
- Novelty detection
- Intrusion detection
- Medical abnormality screening
- Small/medium tabular anomaly detection

12. Common Mistakes

- Not scaling features
- Training on contaminated data
- Misunderstanding `nu`
- Using RBF SVM on huge datasets
- Not tuning `gamma`
- Reporting accuracy on imbalanced data
- Confusing outlier detection and novelty detection

13. Edge Cases / Limitations

One-class SVM is sensitive to scaling, `gamma`, and `nu`. It can be slow on large data and brittle when normal data has many modes.

14. Variations

Variation	What Changes	When To Use	Importance
RBF One-class SVM	Nonlinear boundary	Small/medium nonlinear normal region	High
Linear One-class SVM	Linear boundary	Large sparse data	Medium
SGD One-class SVM	Approximate scalable training	Large data	Medium
SVDD	Minimum enclosing hypersphere	Similar novelty detection	Research

15. Related Topics

One-class SVM relates to standard SVM but uses only one class. It competes with Isolation Forest, LOF, and autoencoder anomaly detection. Isolation Forest is usually easier to scale.

16. Interview Questions

1. What is One-class SVM? A model that learns a boundary around normal data.
2. Is it supervised? Usually unsupervised/semi-supervised with normal-only data.
3. What does `nu` mean? It controls allowed outlier fraction and support vector lower bound.
4. Why use RBF kernel? To learn nonlinear boundaries.
5. Why scale data? SVM kernels depend on distances.
6. What does negative prediction mean in sklearn? Anomaly.
7. One-class SVM vs Isolation Forest? SVM learns a kernel boundary; Isolation Forest uses random isolation and scales better.
8. When avoid One-class SVM? Very large datasets or poorly scaled noisy features.
9. What is novelty detection? Detecting unseen abnormal data after training on normal data.
10. How tune threshold? Use validation anomalies, cost, or alert budget.

17. Practice Tasks

- Train One-class SVM on normal synthetic data.
- Plot decision boundary in 2D.
- Tune `nu` and `gamma`.
- Compare with Isolation Forest.
- Evaluate with PR-AUC on imbalanced data.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Defect Novelty Detector	Flags abnormal product measurements	sklearn	UCI quality data	Manufacturing ML

Project	What It Does	Tech Stack	Dataset	Resume Value
Login Behavior Detector	Finds unusual login sessions	pandas, sklearn	Auth logs	Security analytics
Medical Screening Baseline	Flags abnormal patient records	sklearn	UCI medical datasets	Healthcare ML

19. Quick Revision

- Key idea: learn boundary around normal data.
- Main formula: maximize separation from origin in feature space.
- When to use: normal-only novelty detection.
- Metrics: precision, recall, PR-AUC.
- Common traps: no scaling, bad ν/γ .
- Interview one-liner: One-class SVM detects anomalies by learning the support boundary of normal data.

20. Final Cheat Sheet

Item	Summary
Definition	SVM-based novelty/anomaly detector
Input/output	normal training data -> normal/anomaly decision
Main steps	scale, fit boundary, score new points
Hyperparameters	kernel, ν , γ
Metrics	precision, recall, F1, PR-AUC
Pros	strong nonlinear boundary for small data
Cons	slow, sensitive to scaling/tuning
Best use	clean normal-only novelty detection